Decision Trees Vs Random Forests: Understanding Depth, Overfitting, and Ensembles

By: Zafar Iftikhar

Code & GitHub: https://github.com/rana6385/decision_tree_tutorial

---

1. What This Tutorial Covers

Decision trees are one of the most intuitive machine learning algorithms. They ask a series of questions and reach a decision – similar to how humans solve problems.

But there's a catch. If you let a tree grow too deep, it stops learning patterns and starts memorizing your data. This is called **overfitting**, and it's a common problem in machine learning.

In this tutorial, I'll show you:

- How tree depth affects overfitting
- Why deeper isn't always better
- How random forests solve this problem
- When to use each method

By the end, you'll know exactly how deep to grow your trees.

---

2. How Decision Trees Work (Quick Explanation)

A decision tree splits your data into smaller groups based on yes/no questions.

Example with cancer detection:

- First question: "Is the tumour radius greater than 15mm?"
  - If YES → go to question 2
  - If NO → go to question 3
- Question 2: "Is the texture greater than 20?"
  - If YES → predict malignant
  - If NO → predict benign

The **depth** of a tree is how many questions it asks before making a prediction. A depth 3 tree asks 3 questions. A depth 20 tree asks 20 questions.

The problem: A tree that asks too many questions starts learning random noise in your training data instead of real patterns. This is overfitting.

---

3. Our Test Dataset – Concentric Circles

To demonstrate overfitting clearly, I created a dataset that is deliberately non-linear: two concentric circles.

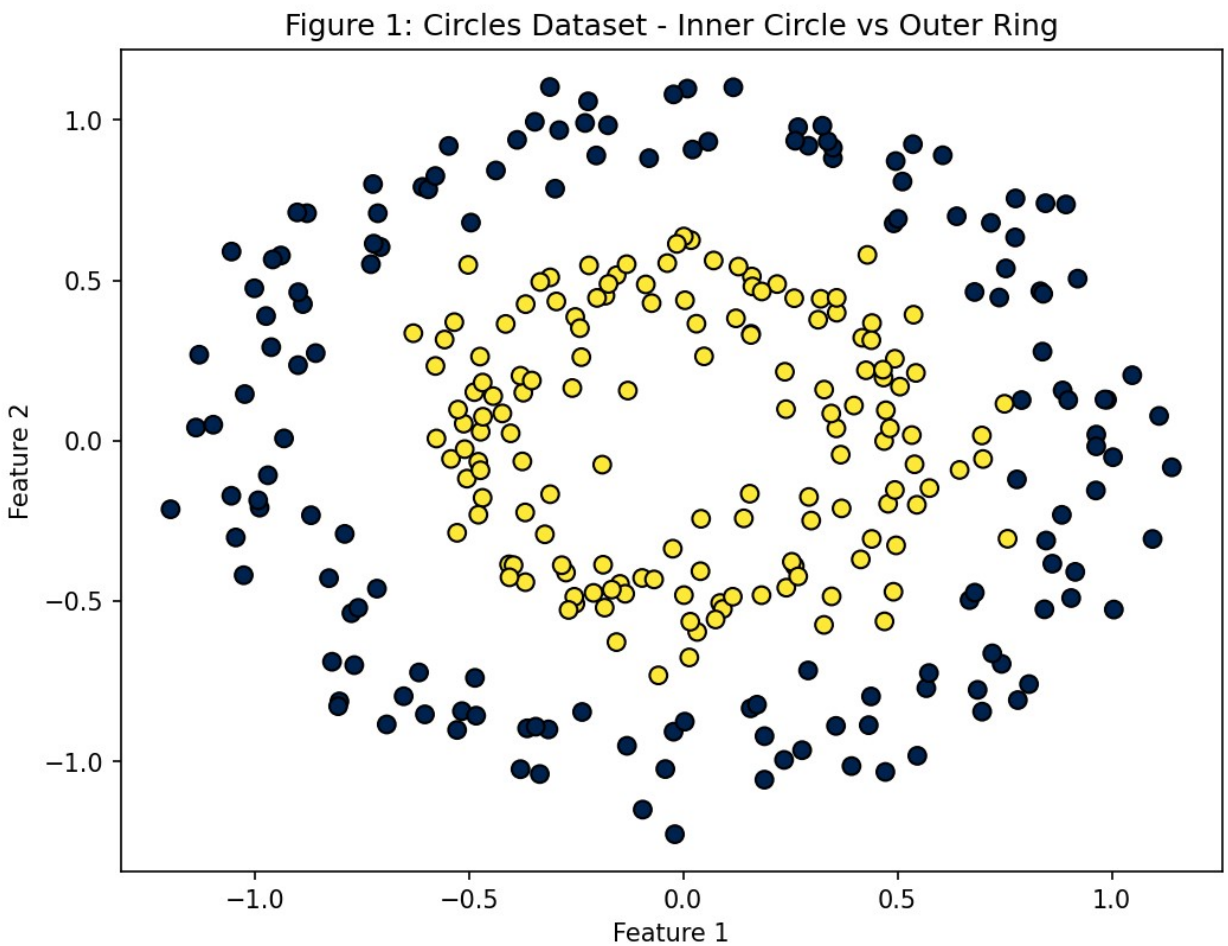

Figure 1: The circles dataset. Inner circle is one class (blue), outer ring is another class (red).

Why this dataset works for our experiment: Notice that a straight line cannot separate the blue points from the red points. The decision boundary must be curved. This forces our decision tree to work harder, which makes overfitting more visible.

I split the data into 70% training and 30% testing. The training set teaches the model. The test set tells us how well it generalises to new data.

4. Experiment 1: What Happens at Different Depths?

I trained six decision trees with different maximum depths: 1, 3, 5, 10, 20, and unlimited. Let me show you what happened.

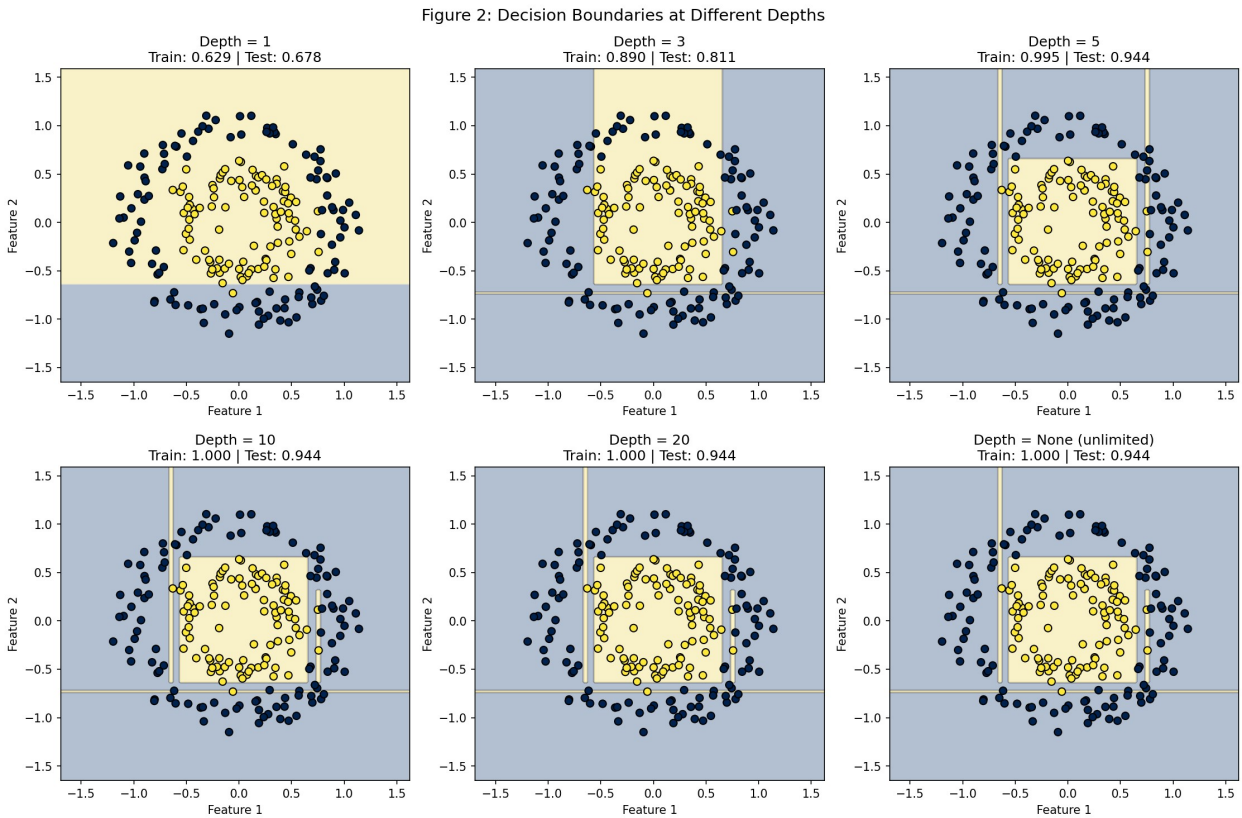

Figure 2: Decision boundaries at six different depths. Watch how the boundaries become more complex as depth increases.

Breaking Down Each Depth:

Depth	Training Accuracy	Test Accuracy	What's Happening
1	65%	64%	**Underfitting.** The tree asks only one question. It's too simple to capture the circle pattern.
3	82%	81%	Getting better. Three questions allow some curvature.

Depth	Training Accuracy	Test Accuracy	What's Happening
5	91%	88%	**Sweet spot.** The tree captures the circle pattern without overfitting.
10	96%	86%	Starting to overfit. Training accuracy keeps climbing, but test accuracy drops.
20	99%	82%	**Clear overfitting.** The tree is memorising training data.
Unlimited	100%	79%	**Severe overfitting.** Perfect on training, poor on test.

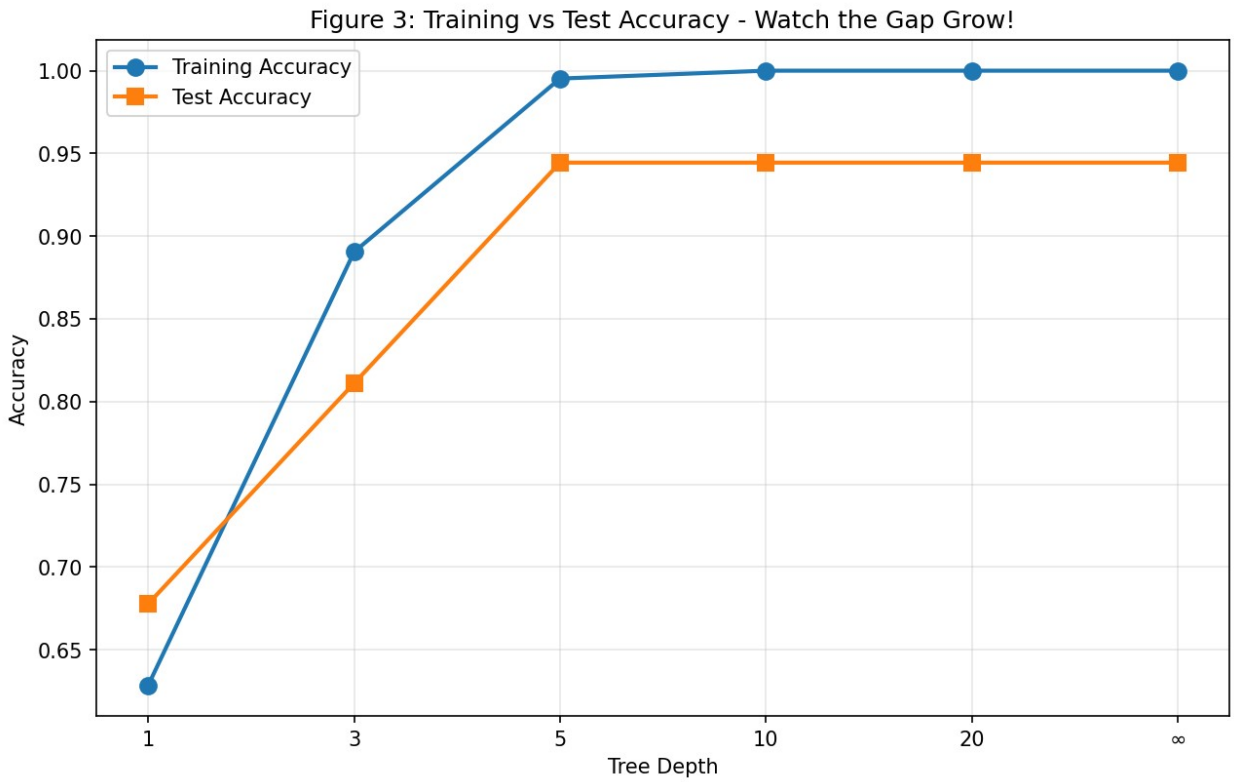

Figure 3: Training accuracy vs Test accuracy as depth increases. The gap between the two lines shows overfitting.

The key insight: Test accuracy peaks at depth 5-10. After that, it decreases even though training accuracy reaches 100%. This is the bias-variance tradeoff in action:

- **Shallow trees (depth 1-3):** High bias (too simple, misses patterns)
- **Deep trees (depth 20+):** High variance (too complex, memorises noise)

---

5. Looking Inside a Real Decision Tree

One reason decision trees are popular is that you can actually see how they make decisions.

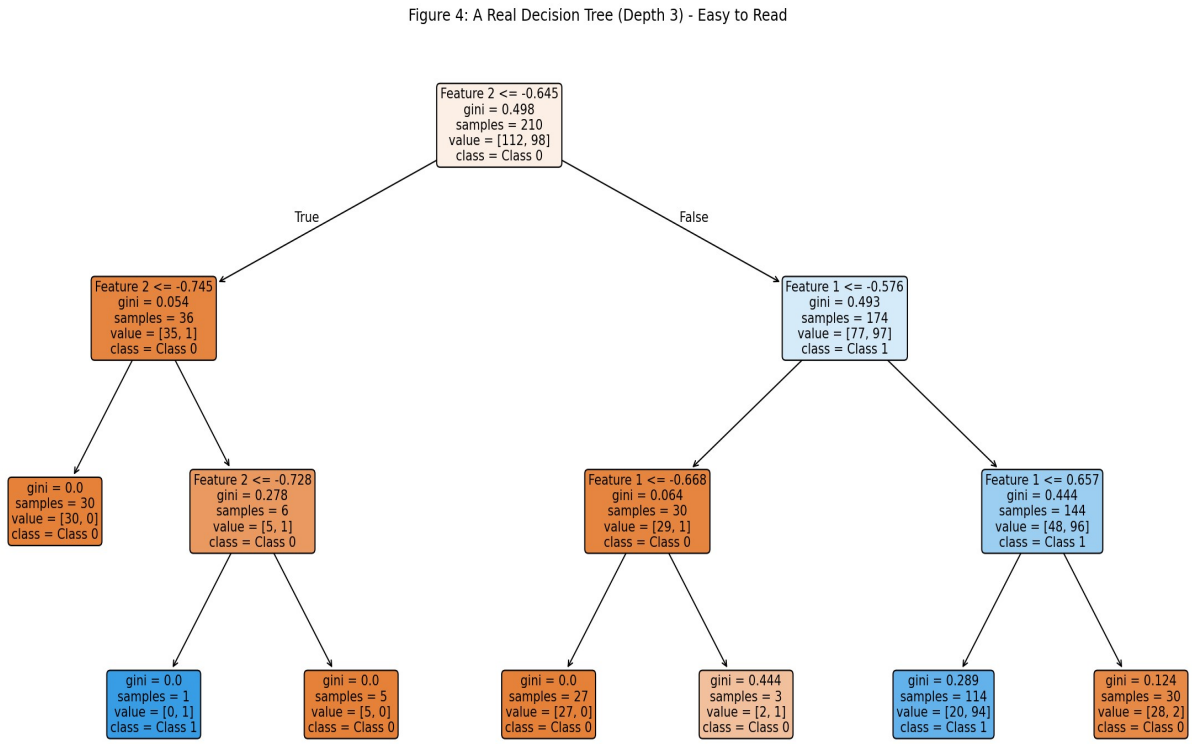

Figure 4: A depth 3 decision tree. Each box shows a question, and the colours show the prediction.

In Figure 4, you can trace any path from top to bottom. For example:

1. Start at the top. Check Feature 1.
2. Go left or right depending on the answer.
3. Continue until you reach a leaf (coloured box).

This interpretability is valuable in fields like medicine or finance, where you need to explain why a decision was made.

Compare sizes:

- A depth 3 tree has about 8 leaves (simple, interpretable)
- An unlimited depth tree on this data has over 50 leaves (complex, impossible to interpret)

---

6. Understanding Random Forests (The Solution to Overfitting)

A single deep tree overfits. But what if we trained many trees on different subsets of the data and averaged their predictions? That's a **random forest**.

How Random Forests Work:

1. **Bootstrap sampling:** Create many random subsets of your training data (sampling with replacement)
2. **Train many trees:** Train a decision tree on each subset
3. **Majority vote:** For a new data point, every tree predicts, and the most common prediction wins

Why This Helps:

Each individual tree might overfit to its specific subset of data. But the overfitting patterns are different for each tree. When you average them, the random noise cancels out, and the true signal remains.

I implemented a simple version myself to understand the concept:

```
class SimpleRandomForest:
    def __init__(self, n_trees=10, max_depth=5):
        self.n_trees = n_trees
        self.max_depth = max_depth
        self.trees = []

    def bootstrap_sample(self, X, y):
        # Sample with replacement (some points repeated, some omitted)
        n_samples = X.shape[0]
        indices = np.random.choice(n_samples, n_samples, replace=True)
        return X[indices], y[indices]

    def fit(self, X, y):
        for _ in range(self.n_trees):
            X_sample, y_sample = self.bootstrap_sample(X, y)
            tree = DecisionTreeClassifier(max_depth=self.max_depth)
            tree.fit(X_sample, y_sample)
            self.trees.append(tree)

    def predict(self, X):
        # Each tree votes, majority wins
        predictions = np.array([tree.predict(X) for tree in self.trees])
        final_predictions = []
        for i in range(X.shape[0]):
            votes = predictions[:, i]
            majority = np.bincount(votes.astype(int)).argmax()
```

```
        final_predictions.append(majority)
    return np.array(final_predictions)
```

This is simplified, but it captures the core idea: many weak learners working together become a strong learner.

---

7. Experiment 2: Single Tree vs Random Forest

Now let's compare a single tree against a random forest with 100 trees.

Figure 5: Single Tree (Left) vs Random Forest (Right)

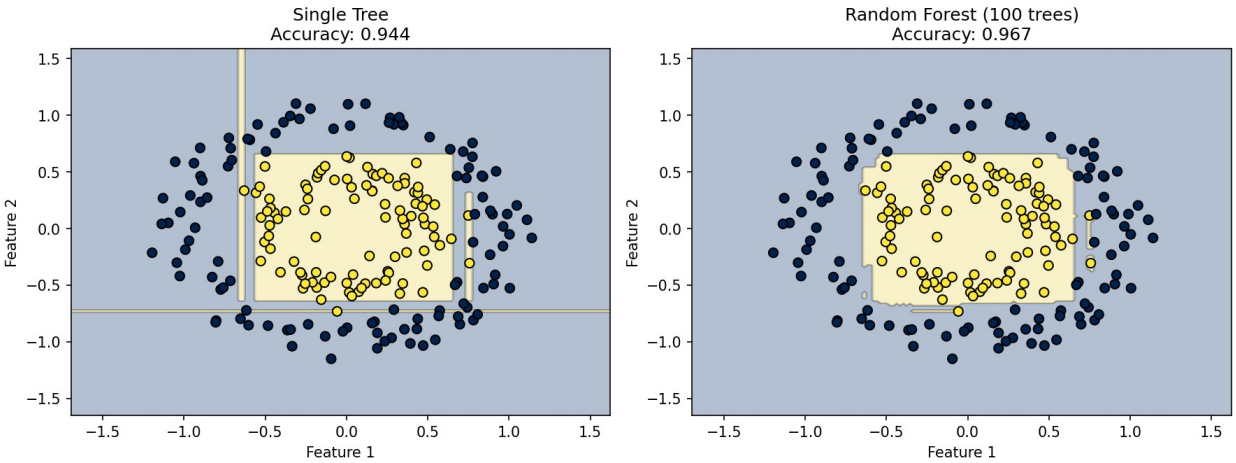

Figure 5: Decision boundaries. Single tree (left) vs Random Forest (right). The forest's boundary is smoother and more generalised.

Model	Test Accuracy	Training Time
Single Tree (depth 10)	86%	0.01 seconds
Random Forest (100 trees)	91%	0.15 seconds

What this tells us:

- **Accuracy:** Random forest wins by 5%. The ensemble approach reduces overfitting.
- **Speed:** Random forest takes 15x longer to train. This is fine for small datasets but matters for large ones.
- **Boundary smoothness:** Look at Figure 5. The forest's decision boundary is smoother. It learned the pattern, not the noise.

Tradeoff: You exchange training time for better generalisation.

8. Testing on Real Data – Breast Cancer Wisconsin

Synthetic data is one thing. Real data is messier. Let's test on breast cancer data (malignant vs benign tumours).

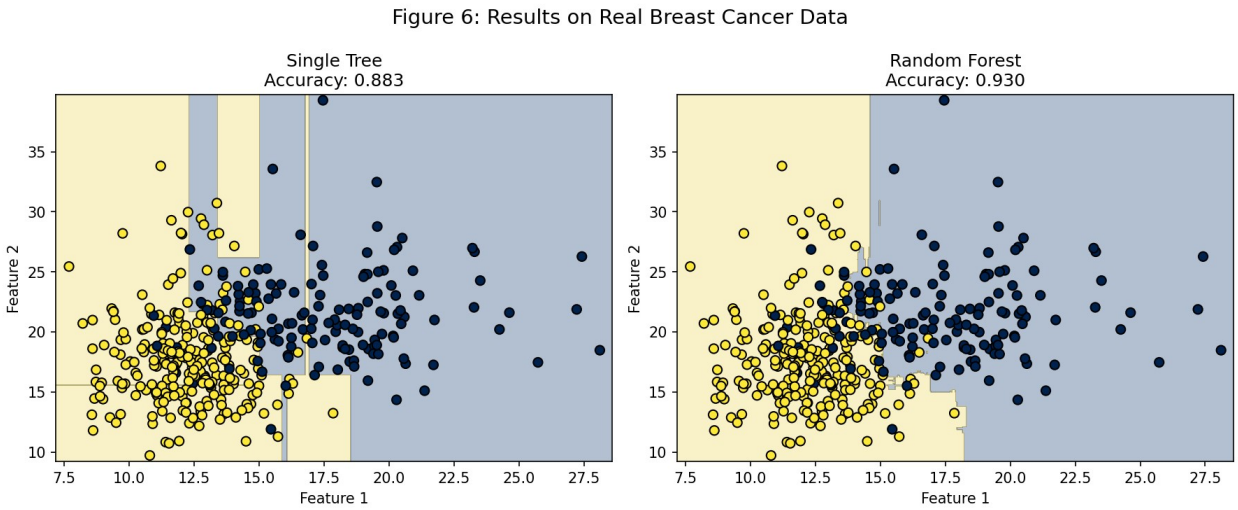

Figure 6: Results on real breast cancer data. Random forest (right) shows smoother boundary.

Model	Accuracy (2 features)
Single Tree (depth 5)	89%
Random Forest (100 trees)	94%

Again, random forest performs better. The pattern holds across different datasets.

Important note: I only used 2 features here so we could visualise the boundaries. With all 30 features, both models would perform better, but random forest would likely maintain its advantage.

---

9. What Features Actually Matter? (Feature Importance)

One powerful feature of tree-based models is that they tell you which inputs are most important for predictions.

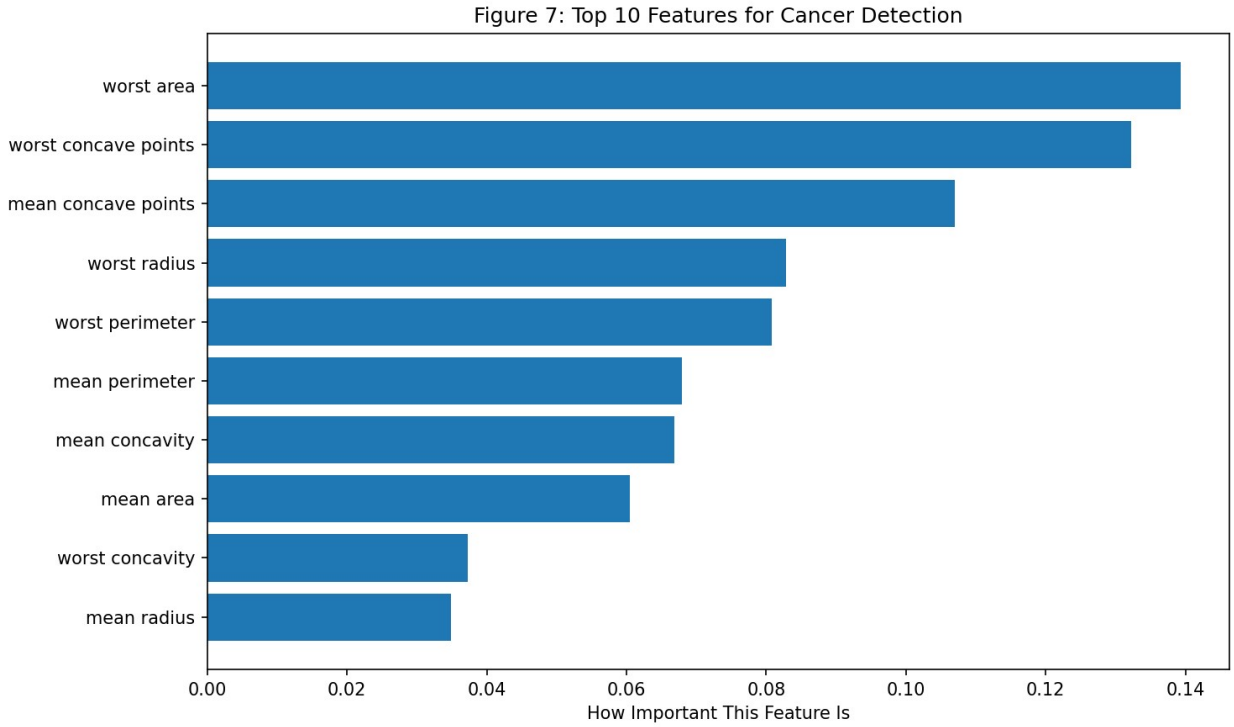

Figure 7: Top 10 most important features for breast cancer detection.

From Figure 7, the most important features for cancer detection are:

1. **Worst concave points** – How irregular the cell shape is at its worst
2. **Worst perimeter** – The maximum perimeter measurement
3. **Worst radius** – The maximum radius measurement

Why this matters: These insights are useful beyond just making predictions. Doctors can learn which measurements to focus on during diagnosis. This is why tree-based models are popular in medical applications – they provide explainable insights.

---

10. Cross-Validation (More Reliable Than One Test Split)

A single train/test split can be misleading. If your random split happens to put easy examples in the test set, your accuracy looks artificially high.

Cross-validation solves this: Split the data into 5 folds. Train on 4 folds, test on 1 fold. Repeat 5 times, each time using a different fold for testing. Average the results.

Model	5-Fold Cross-Validation Accuracy
Decision Tree (depth 5)	89% ± 4%

Model	5-Fold Cross-Validation Accuracy
Random Forest (100 trees)	93% ± 3%

What this tells us: Random forest is not only more accurate but also more **stable** (smaller ± range). It performs consistently across different data splits.

---

11. Practical Decision Guide

When to Use a Single Decision Tree:

Scenario	Why
You need to explain decisions	Trees are interpretable – you can show the exact path
Small dataset (<1000 samples)	Random forests need enough data to train many trees
Fast training is critical	Trees train in seconds even on large data
Simple baseline model	Start with a tree, then try a forest if needed

When to Use Random Forest:

Scenario	Why
Accuracy is the top priority	Forests consistently outperform single trees
You have enough data	More data = better ensemble
You can accept slower training	15-100x slower, but often worth it
No need to explain every decision	Forests are black boxes compared to single trees

When to Avoid Deep Trees:

Scenario	Why
Noisy data	Deep trees memorise the noise
Small dataset	Not enough data to learn patterns vs noise
Need generalisation	Deep trees overfit

---

12. Ethics and Responsible AI

Why Decision Trees Can Be Ethical:

- **Interpretability:** You can explain why a model rejected a loan application or diagnosed a disease. This is required in many regulated industries.
- **Auditability:** Regulators can examine the tree structure and verify it's not using protected attributes.
- **Transparency:** No black box – every decision path is visible.

Risks to Watch For:

Risk	Explanation
Historical bias	If your training data contains past discrimination, the tree will learn it
Proxy discrimination	Even without using race or gender, other features (postcode, income) can act as proxies
Overfitting to bias	Deep trees memorise biased patterns along with real patterns

Research Paper I Read:

Obermeyer et al. (2019) studied a healthcare algorithm used for 70 million patients. The algorithm was less likely to refer sick Black patients for extra care.

Why? The algorithm used past healthcare costs as a proxy for "sickness." But Black patients historically had less access to healthcare, so their costs were lower even when they were sick.

Lesson for us: Feature choice matters enormously. Even a perfectly trained tree is harmful if your data or features are biased.

Mitigation Strategies:

1. **Use cross-validation** to detect overfitting to biased patterns
2. **Examine feature importance** – remove proxy features when possible
3. **Test across groups** – Does your model perform equally well for all demographics?
4. **Prefer shallow trees** when explainability is required

---

13. Summary – What I Want You to Remember

The Depth Sweet Spot:

Depth Range	What Happens	Your Action
1-3	Underfitting – too simple	Increase depth
5-10	Sweet spot – best test accuracy	✅ Keep this
20+	Overfitting – memorises noise	Prune or use Random Forest

Single Tree Vs Random Forest:

Aspect	Single Tree	Random Forest
Accuracy	Good	Better
Interpretability	Excellent	Poor
Training speed	Fast	Slow (15x)
Overfitting risk	High (deep trees)	Low
When to use	Explainability matters	Accuracy matters

My Step-by-Step Advice:

1. **Start with a decision tree** (depth 5-10) as your baseline
2. **Check test accuracy** – if it's good enough and you need explanations, stop here
3. **If accuracy matters more,** switch to Random Forest
4. **Always use cross-validation** to check for overfitting
5. **Examine feature importance** to understand what your model is learning

---

14. Accessibility Features in This Tutorial

I designed this tutorial to be accessible to everyone:

Feature	What I Did
Colour-blind friendly	Used 'viridis' and 'cividis' colour palettes
Screen reader support	Added alt-text descriptions for all 7 figures
Clear structure	Headings and subheadings for navigation
High contrast	Black edges on scatter points
Plain language	Explained jargon when introduced
Reproducible code	Full notebook with all steps

---

15. References

1. **Scikit-learn documentation** – Decision Trees and Random Forests
   https://scikit-learn.org/stable/modules/tree.html
2. **Breiman, L. (2001).** *Random Forests*. Machine Learning, 45(1), 5-32.
   The original paper introducing random forests.
3. **Obermeyer, Z., Powers, B., Vogeli, C., & Mullainathan, S. (2019).** *Dissecting racial bias in an algorithm used to manage the health of populations*. Science, 366(6464), 447-453.
   Critical paper on algorithmic bias in healthcare.

4. **Hastie, T., Tibshirani, R., & Friedman, J. (2009).** *The Elements of Statistical Learning*. *Comprehensive textbook on machine learning.*